

ЗВІТ - ЛБ4 - ОПІ - Конишев Денис - ПЗПІ-25-1

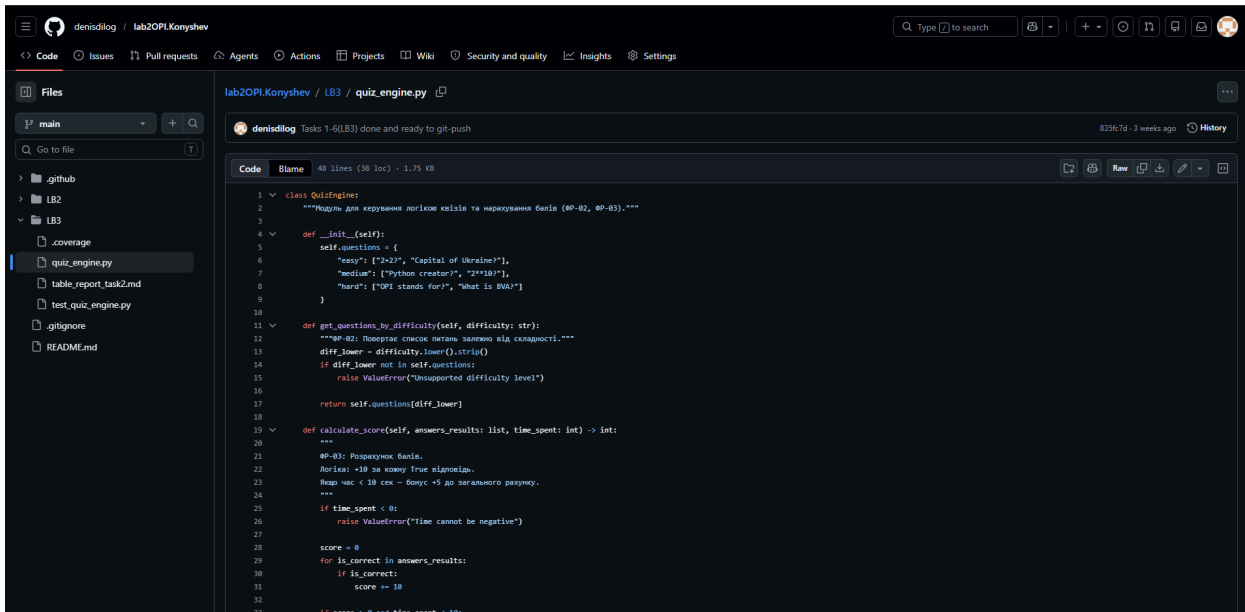
РЕФАКТОРИНГ КОДУ ТА ПРОВЕДЕННЯ CODE REVIEW

Мета роботи: набуття практичних навичок із проведення Code Review за індустріальними стандартами з використанням GitHub Pull Requests. Оволодіння методами ідентифікації та класифікації типових проблем якості коду (code smells). Отримання досвіду застосування рефакторинг-операцій із верифікацією збереження поведінки через регресійне тестування.

[Гітхаб репозиторій](#)

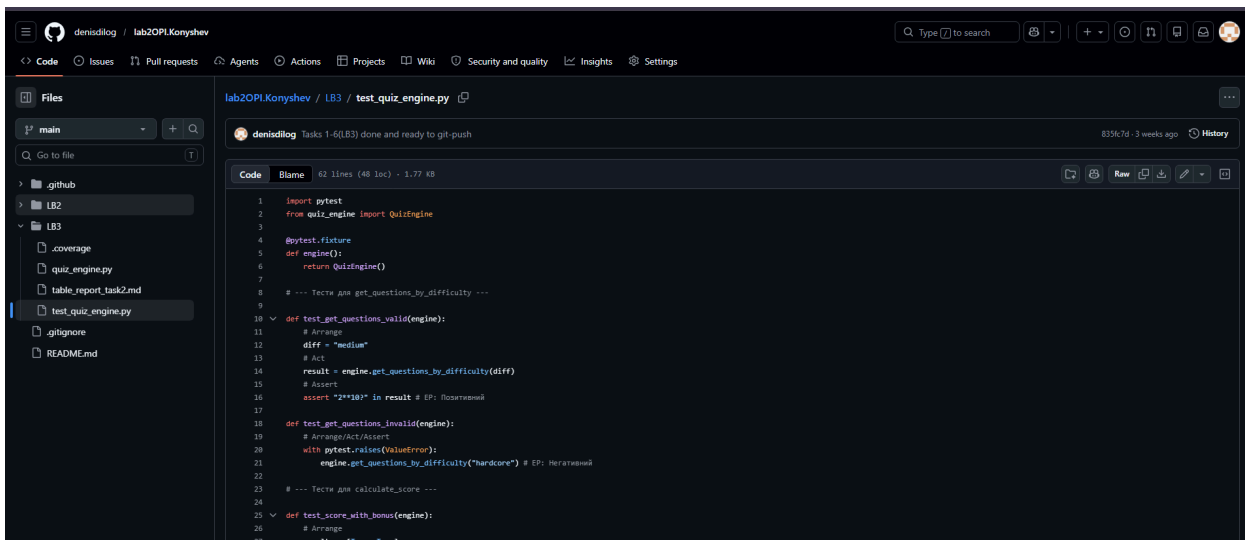
ЕТАП 1

крок1



The screenshot shows the GitHub web interface for the repository 'denisidlog / lab2OPLKonyshov'. The file 'quiz_engine.py' is selected, showing 48 lines of Python code. The code defines a 'QuizEngine' class with methods for initializing questions, getting questions by difficulty, and calculating scores. Comments in Ukrainian are interspersed throughout the code.

```
1 class QuizEngine:
2     """Клас для керування набором питань та нарахування балів (0-10, 0-10)."""
3
4     def __init__(self):
5         self.questions = {
6             "easy": [{"id": 1, "question": "Capital of Ukraine?"},
7                     {"id": 2, "question": "Python creator?"}, {"id": 3, "question": "What is RWA?"}],
8             "medium": [{"id": 4, "question": "What is RWA?"}],
9             "hard": [{"id": 5, "question": "What is RWA?"}]
10        }
11
12    def get_questions_by_difficulty(self, difficulty: str):
13        """Get questions by difficulty level. Returns a list of questions.
14        If difficulty is not supported, raises ValueError.
15        """
16        if difficulty.lower() not in self.questions:
17            raise ValueError("Unsupported difficulty level")
18        return self.questions[difficulty.lower()]
19
20    def calculate_score(self, answers_results: list, time_spent: int) -> int:
21        """Calculate score based on answers and time spent.
22        Returns a score from 0 to 10.
23        """
24        score = 0
25        for is_correct in answers_results:
26            if is_correct:
27                score += 10
28
29        if score > 10 and time_spent < 10:
```



The screenshot shows the GitHub web interface for the repository 'denisidlog / lab2OPLKonyshov'. The file 'test_quiz_engine.py' is selected, showing 22 lines of Python code. The code uses pytest to test the 'QuizEngine' class methods, including assertions for question counts and score calculations.

```
1 import pytest
2 from quiz_engine import QuizEngine
3
4 @pytest.fixture
5 def engine():
6     return QuizEngine()
7
8 # --- Test get_questions_by_difficulty ---
9
10 def test_get_questions_valid(engine):
11     # Arrange
12     diff = "medium"
13     # Act
14     result = engine.get_questions_by_difficulty(diff)
15     # Assert
16     assert len(result) == 1, "Expected 1 question"
17
18 def test_get_questions_invalid(engine):
19     # Arrange/Act/Assert
20     with pytest.raises(ValueError):
21         engine.get_questions_by_difficulty("hardcore") # EP: Hardcore
22
23 # --- Test calculate_score ---
24
25 def test_score_with_bonus(engine):
26     # Arrange
27     results = [True, True]
```

EXTENSIONS: ...

SonarLint

Angular Dais... 318
The Essentials Extensio...
Gyduhnhn

Install

Angular Prim... 1K
The Essentials Extensio...
Gyduhnhn

Install

Essentials Ext... 8K
The Essentials Extensio...
Gyduhnhn

Install

Essentials Ext... 3K
The Essentials Extensio...
Gyduhnhn

Install

Angular NG-... 132
The Essentials Extensio...
Gyduhnhn

Install

SonarQub... 1524ms
Advanced linter to det...
SonarSource

Install

quiz_engine.py 3

Extension: SonarQube for IDE

4,419,262

★★★★☆ (122)

Advanced linter to detect & fix coding issues locally in JS/TS, Python, Java, C#, C/C++, Go, PHP. Use with SonarQube (Server, Cloud) for optimal team performance.

Disable

Uninstall

☒ Auto Update

DETAILS

FEATURES

CHANGELOG

Installation

Identifier

sonarsource.sonarlint-vscode

Version

5.3.0

Last Updated

19 minutes ago

Size

319.51MB

Marketplace

Published

9 years ago

PROBLEMS 3

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

SONARQUBE 3

EXPLORER

OPEN EDITORS

quiz_engine.py 3

test_quiz_engine.py

UNTITLED (WORKSPACE)

lab2OPI.Konyshov

github\workflows

tests.yml

LB2

LB3

__pycache__

.pytest_cache

htmlcov

.coverage

quiz_engine.py 3

table_report_task2...

test_quiz_engine.py

.gitignore

README.md

lab2OPI.Konyshov > LB3 > quiz_engine.py > QuizEngine > determine_rank

1 import os

2 import math

3

4 class QuizEngine:

5 """Модуль для керування логікою квізів та нарахування балів (ФР-02, ФР-03)."""

6

7 def __init__(self):

8 # old questions = ["test1?", "test2?"]

9 self.questions = {

10 "easy": ["2+2?", "Capital of Ukraine?"],

11 "medium": ["Python creator?", "2**10?"],

12 "hard": ["OPI stands for?", "What is BVA?"]

13 }

14

15 def get_questions_by_difficulty(self, difficulty: str):

16 """ФР-02: Повертає список питань залежно від складності."""

17 d = difficulty.lower().strip()

18 if d == "easy":

19 return self.questions["easy"]

20 elif d == "medium":

21 return self.questions["medium"]

22 elif d == "hard":

23 return self.questions["hard"]

24

PROBLEMS 3

OUTPUT

DEBUG CONSOLE

TERMINAL

PORTS

SONARQUBE 3

SONARQUBE FINDINGS (ALL FINDINGS)

3

quiz_engine.py LB3\

Remove this commented out code. (python:S125) [Ln 8, Col 8]

Remove this commented out code. (python:S125) [Ln 40, Col 8]

Merge this if statement with the enclosing one. [+1 location] (python:S1066) [Ln 42, ...]

SONARQUBE FOR IDE LABS

Be a Sonar Insider

SonarQube

крок3 - Ruff для Python

```
PS I:\дискД (19.02.2026)после сбоя матрицы\Денис\ХНУРЕ\Учеба(второй семестр_первый курс)\ОПІ\Lab2\lab20PI.Konyshev> ruff check LB3/
F401 [*] `os` imported but unused
--> LB3\quiz_engine.py:1:8
1 | import os
  |         ^^
2 | import math
  |
help: Remove unused import: `os`

F401 [*] `math` imported but unused
--> LB3\quiz_engine.py:2:8
1 | import os
2 | import math
  |         ^^^^
3 |
4 | class QuizEngine:
  |
help: Remove unused import: `math`

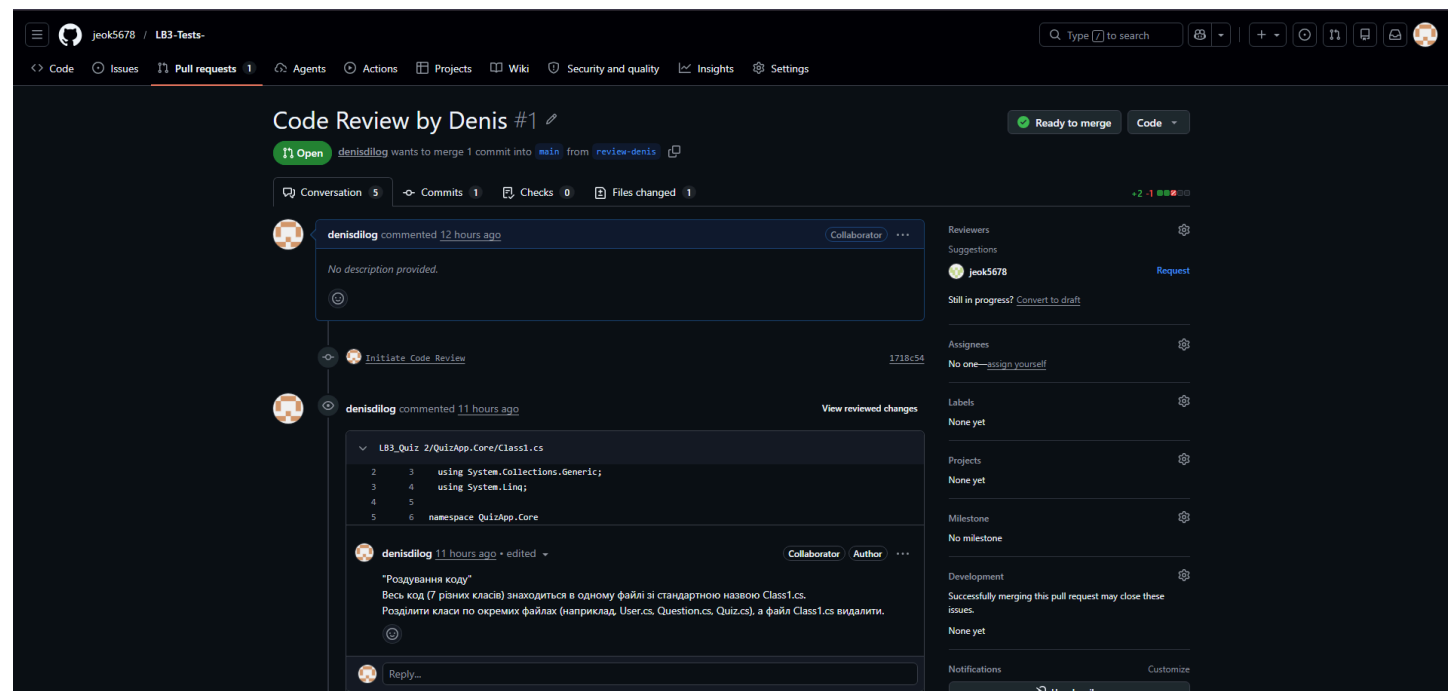
E712 Avoid equality comparisons to `True`; use `ar[i]:` for truth checks
--> LB3\quiz_engine.py:37:16
35 |         s = 0
36 |         for i in range(len(ar)):
37 |             if ar[i] == True:
  |                 ^^^^^^^^^^^^^
38 |                 s = s + 10
  |
help: Replace with `ar[i]`

Found 3 errors.
[*] 2 fixable with the `--fix` option (1 hidden fix can be enabled with the `--unsafe-fixes` option).
```

ЕТАП 2

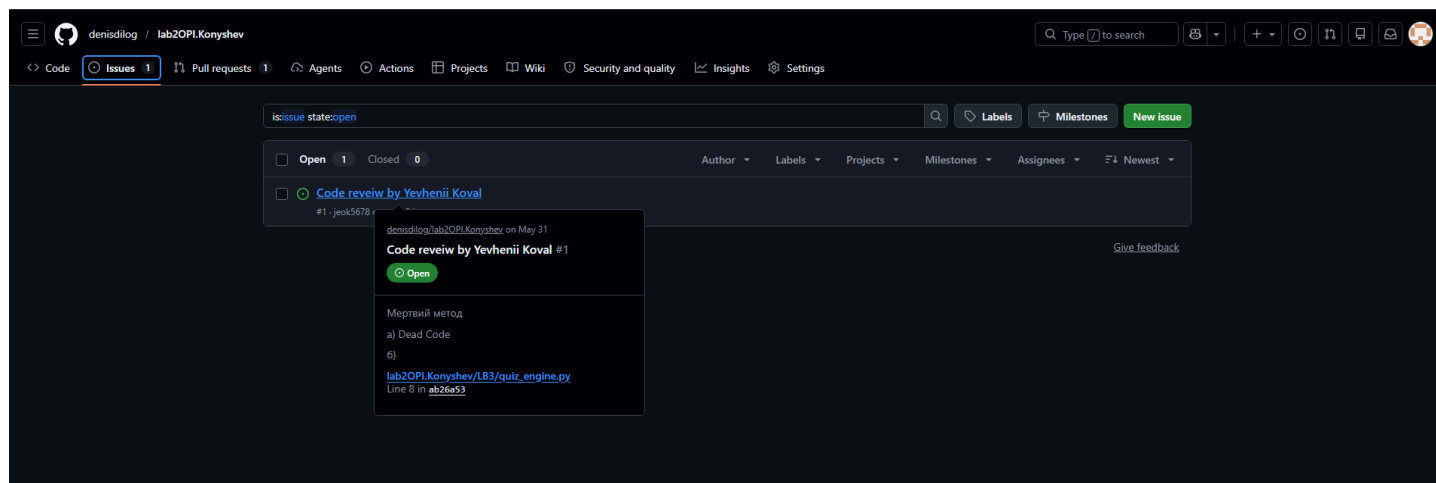
Pull Request в репозиторії одногрупника:

<https://github.com/jeok5678/LB3-Tests-/pull/1>



ЕТАП 3

крок1 - отримав ревью від одnogрупника



крок2 - перша рефакторинг операція до рефакторингу тести - пройдено

```
PS I:\дискД (19.02.2026)после сбоя матрицы\Денис\ХНУРЕ\Учеба(второй семестр_первый курс)\ОПИ\Lab2\lab2OPI.Konyshev> pytest
===== test session starts =====
platform win32 -- Python 3.10.2, pytest-9.0.3, pluggy-1.6.0
rootdir: I:\дискД (19.02.2026)после сбоя матрицы\Денис\ХНУРЕ\Учеба(второй семестр_первый курс)\ОПИ\Lab2\lab2OPI.Konyshev
plugins: cov-7.1.0
collected 10 items

LB3\test_quiz_engine.py ..... [100%]

===== 10 passed in 1.16s =====
```

Операція 1 - Видалення мертвого коду та невикористаних імпортів

БУЛО: Рядок `import os` та закоментовані фрагменти `# old_questions = ["test1?", "test2?"]`

СТАЛО: Ці рядки повністю видалені з коду.

ЧОМУ: Покращення читабельності коду. Закоментований код засмічує проєкт (Dead Code), а невикористані імпорти (Unused Imports) марно споживають пам'ять та можуть викликати попередження лінера. Історія змін все одно зберігається у Git.

тести - пройдено

```
PS I:\дискД (19.02.2026)после сбоя матрицы\Денис\ХНУРЕ\Учеба(второй семестр_первый курс)\ОПИ\Lab2\lab2OPI.Konyshev> pytest
===== test session starts =====
platform win32 -- Python 3.10.2, pytest-9.0.3, pluggy-1.6.0
rootdir: I:\дискД (19.02.2026)после сбоя матрицы\Денис\ХНУРЕ\Учеба(второй семестр_первый курс)\ОПИ\Lab2\lab2OPI.Konyshev
plugins: cov-7.1.0
collected 10 items

LB3\test_quiz_engine.py ..... [100%]

===== 10 passed in 0.50s =====
PS I:\дискД (19.02.2026)после сбоя матрицы\Денис\ХНУРЕ\Учеба(второй семестр_первый курс)\ОПИ\Lab2\lab2OPI.Konyshev>
```

Операція 2 - Заміна "магічних чисел" на константи

БУЛО: $s = s + 10$ та $s = s + 5$

СТАЛО: `POINTS_PER_ANSWER = 10`, `SPEED_BONUS = 5`, $s = s + \text{POINTS_PER_ANSWER}$...

ЧОМУ: Усунення Code Smell "Magic Numbers". Використання іменованих констант робить бізнес-логіку нарахування балів прозорою та полегшує подальшу зміну цих значень у майбутньому (в одному місці).

тести - пройдено

```
PS I:\дискД (19.02.2026)после сбоя матрицы\Денис\ХНУРЕ\Учеба(второй семестр_первый курс)\ОПИ\Lab2\lab2OPI.Konyshev> pytest
===== test session starts =====
platform win32 -- Python 3.10.2, pytest-9.0.3, pluggy-1.6.0
rootdir: I:\дискД (19.02.2026)после сбоя матрицы\Денис\ХНУРЕ\Учеба(второй семестр_первый курс)\ОПИ\Lab2\lab2OPI.Konyshev
plugins: cov-7.1.0
collected 10 items

LB3\test_quiz_engine.py ..... [100%]

===== 10 passed in 0.95s =====
```

Операція 3 - Оптимізація обходу колекції та умов

БУЛО: `for i in range(len(ar)):` та зайва перевірка `if sc < 100:` перед `elif sc >= 50:`.

СТАЛО: Використано ідіоматичний цикл `for answer in ar:`, а перевірки спрощено до послідовних `if ... elif ...`.

ЧОМУ: Прямий обхід колекції у Python працює швидше і є стандартом (PEP 8).

Видалення надлишкових умов (Redundant Conditions) зменшує цикломатичну складність та робить логіку простішою для розуміння.

тести - пройдено

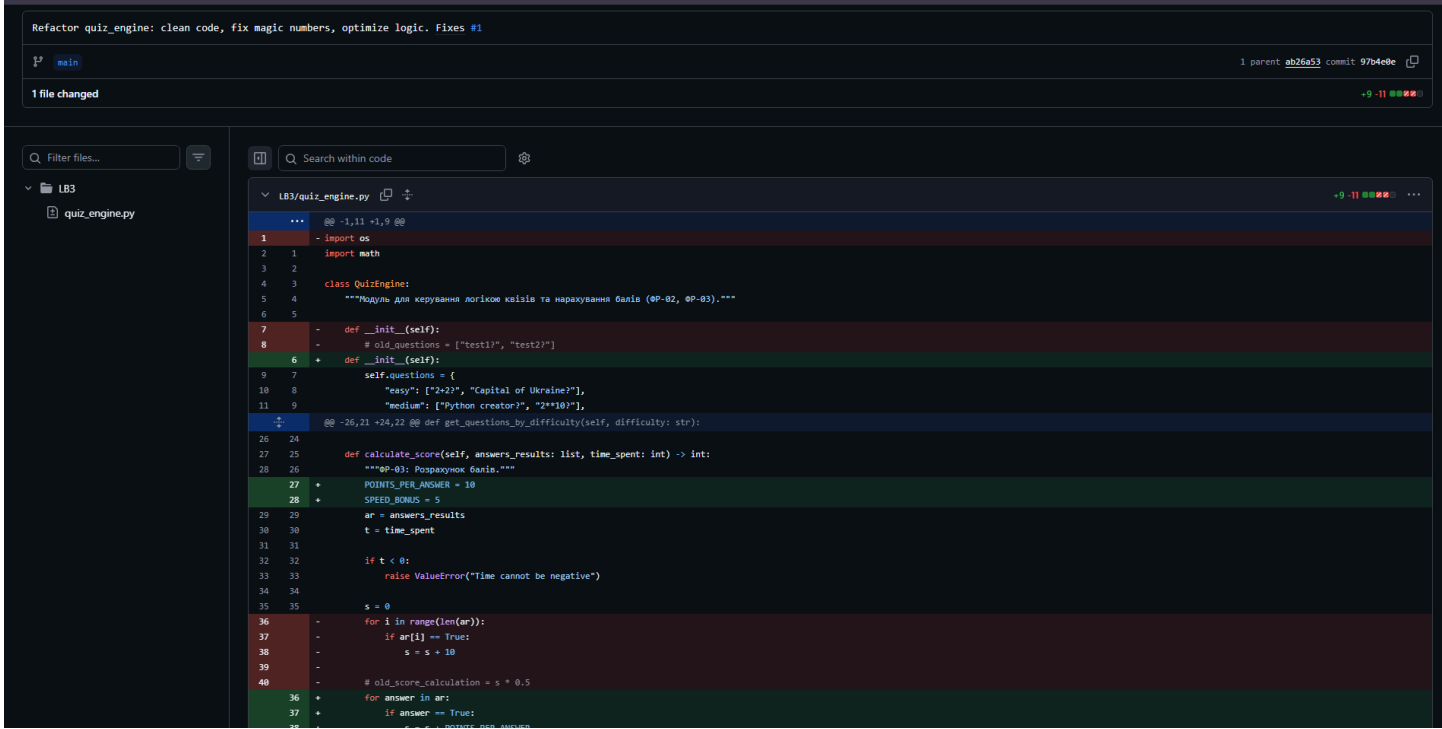
```
===== test session starts =====
platform win32 -- Python 3.10.2, pytest-9.0.3, pluggy-1.6.0
rootdir: I:\дискД (19.02.2026)после сбоя матрицы\Денис\ХНУРЕ\Учеба(второй семестр_первый курс)\ОПИ\Lab2\lab2OPI.Konyshev
plugins: cov-7.1.0
collected 10 items

LB3\test_quiz_engine.py ..... [100%]

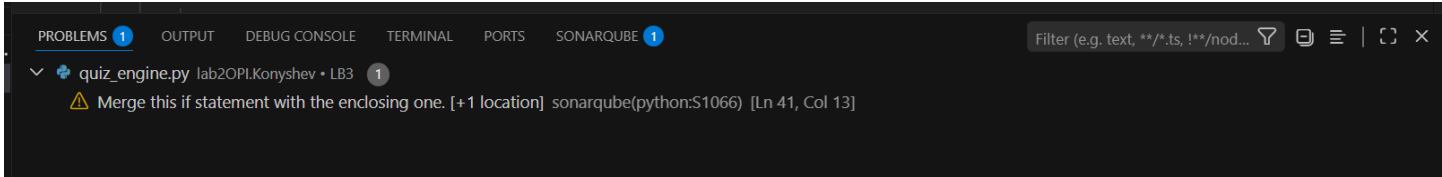
===== 10 passed in 0.29s =====
PS I:\дискД (19.02.2026)после сбоя матрицы\Денис\ХНУРЕ\Учеба(второй семестр_первый курс)\ОПИ\Lab2\lab2OPI.Konyshev>
```

ЕТАП 4

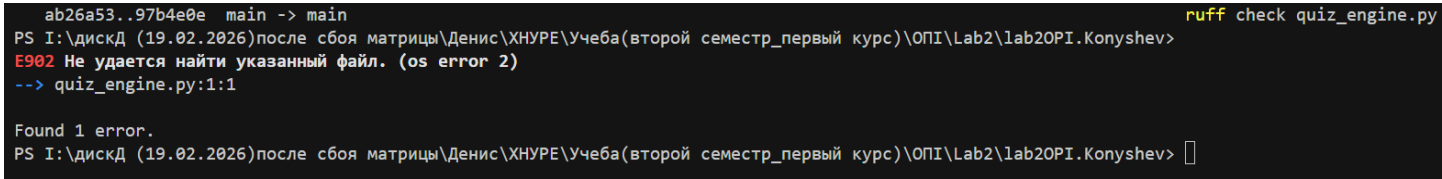
проведено рефакторинг операції



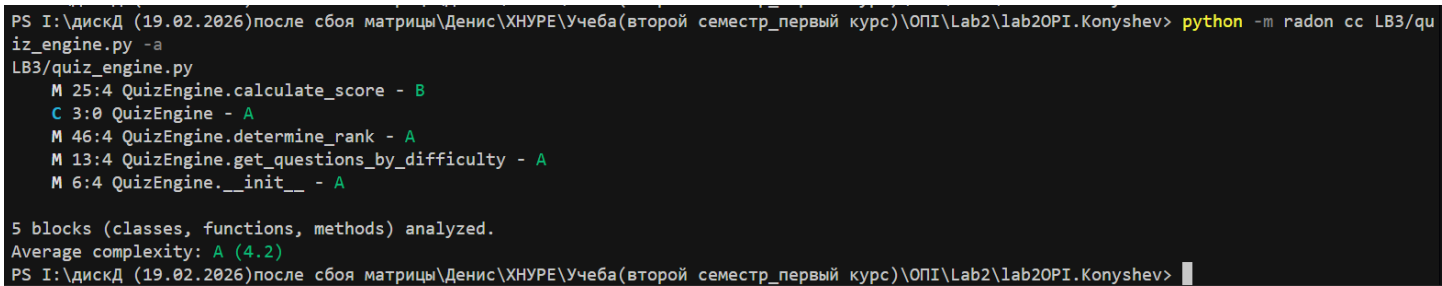
крок1



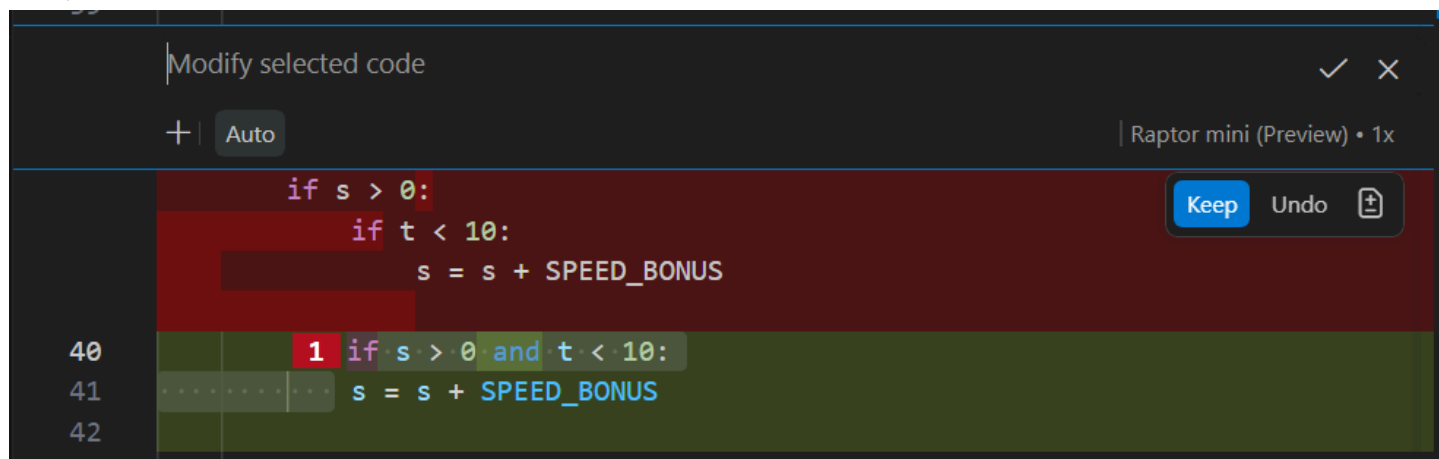
крок2



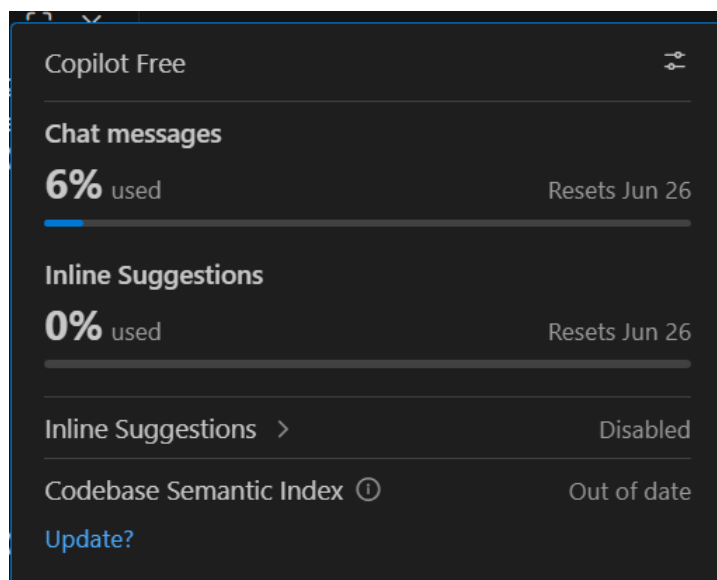
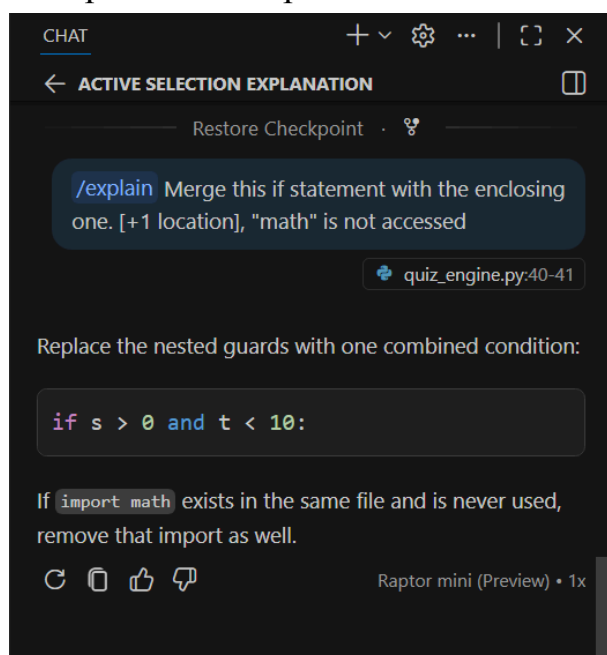
крок3



бонусне завдання*



Використання Copilot Student



Підсумок: найціннішим досвідом під час Code Review стала можливість подивитися на свій код очима іншого розробника. Я зрозумів, що речі, які здаються очевидними автору (як-от магічні числа), можуть бути незрозумілими іншим. Процес рефакторингу зауважень показав, як дрібні зміни суттєво покращують якість, а наявність автотестів (регресійне тестування) дає впевненість, що ці зміни не зламали існуючий функціонал.